

OSSP-4

Question:

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Solution:

```
ubuntu@ubuntu:~$ nano wait_waitpid.c
ubuntu@ubuntu:~$

GNU nano 8.7.1 wait_waitpid.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid[3];

    for(int i = 0; i < 3; i++) {
        pid[i] = fork();

        if(pid[i] == 0) {
            printf("Child %d started with PID %d\n", i+1, getpid());
            sleep(i + 1);
            printf("Child %d finished\n", i+1);
            exit(0);
        }
    }
}

ubuntu@ubuntu:~$ gcc waitpid_demo.c -o waitpid_demo
ubuntu@ubuntu:~$ ./waitpid_demo
Child 1 PID=5875

Using waitpid():
Child 3 PID=5877
Child 2 PID=5876
Parent collected child PID 5875
Parent collected child PID 5876
Parent collected child PID 5877

ubuntu@ubuntu:~$ nano zombie.c
ubuntu@ubuntu:~$ gcc zombie.c -o zombie
ubuntu@ubuntu:~$ ./zombie &
[1] 5908
Parent PID = 5908
Child Process PID = 5910
Child exiting...
ubuntu@ubuntu:~$
```

```
ubuntu@ubuntu:~$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
1 Z  1000      5910      5908  0  80   0  -   0  -   pts/0        00:00:00 zombie
ubuntu@ubuntu:~$ Parent exiting...
```

```
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$ nano no_zombie.c
```

```
ubuntu@ubuntu:~$ gcc no_zombie.c -o no_zombie
ubuntu@ubuntu:~$ ./no_zombie
Child PID = 5955
Parent collected child process
ubuntu@ubuntu:~$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
ubuntu@ubuntu:~$
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if(pid == 0) {
        printf("Child PID = %d\n", getpid());
        exit(0);
    }
    else {
        wait(NULL);
        printf("Parent collected child process\n");
    }

    return 0;
}
```

Read 20 lines